COMP1009
Programming Paradigms

Lecture OO2 – Java and C,
Using objects

Introductory reminders

- Lecture recordings should automatically appear in the Echo360 section after the lecture
- All Java files are in the last section of the moodle page if you need them all in one place, but also linked to from relevant parts
  - Coursework in coursework section
  - Lecture slides/labs info, any code, etc in the weekly info section
- I suggest to use/install Eclipse IDE for Java developers
- First Lab is on the moodle page – you can do it in labs or on your own
  - It should step you through it
- The first (really simple) coursework is on the moodle page
  - In the coursework section
  - Submit via GIT not moodle
  - Do Lab Exercises 1 first

This lecture

- With any new language it is hard at first
  - Everything is linked together – where do we start?
- We want to apply what we already know
- Practice using some very simple classes
  - String and StringBuffer
- Converting some C code into (procedural) Java
  - Using class methods (static member functions)
  - The String class (immutable strings)
  - The StringBuffer class (mutable strings)
  - Command line arguments
  - Control structures: Conditionals, loops, etc
  - Variables

Java classes

- **Some 'Object Oriented' languages do let you use global functions (e.g., C++, Python)**
  - **They also allow you to easily write non-OO code!**
- **Java doesn't**
- **All** executable code in Java **MUST** be in classes
  - Functions are defined inside classes (i.e. the implementation is in the file defining the class)
  - **So, all executable code is inside a class**
  - **There are no global functions in Java!!!**
- Note: Functions in classes are called 'methods'

Simple Java Text Hello World

```
public class HelloWorld
{
  public static void main(String[] args)
  {
    System.out.println( "Hello World!" );
  }
}
```

- Two things to notice:
  - public – accessible outside the class
  - static – no *object* associated with this function

Conversion from C – class functions

- For this lecture all of our functions will be static
  - Not associated with an object to act on
  - This is bad practice in general!
  - Do not do this after these exercises
  - It leads to procedural/imperative designs, not OO designs
- We will also make everything public
  - Public allows any code anywhere to use the class, call the function, or access public data
  - Private is the most restrictive - allows only code in the same class to call the functions (or access data)
  - Protected and package (default) will be considered later
- The class is public: `public class HelloWorld`
  - **Trust me and make all of your top-level classes public** too (at least for this module), it'll avoid some problems later
  - **Classes then need to be defined in a file of the same name**

Static methods (functions in a class)

```
public static void main( String[] args)
```

- We create main() as a **static** function in a class
- **We can also create other static methods in a class**
  - Methods can call other static methods
  - You can pass parameters
  - **Think of these as the equivalent of global functions in C**
- Using static methods, code is not being executed on a specific object, so not really object oriented programming
- But this is useful at the moment:
  - To show you the huge similarities between Java and C
  - To learn the Java syntax for common features
  - To see how to **use** objects before you make your own

Converting C programs to Java

- We are going to do some bad things
- Writing ‘procedural Java’
  - Not Object Oriented
  - So we can see the C similarities
- Put everything in one class
  - Simplifies things for this exercise
  - Don’t do this with a big program!

Calling a function with a string, in C

```c
#include <stdio.h>

void printMessage( char* message )
{
  printf("%s\n", message);
}

int main(int argc, char* argv[])
{
  printMessage("Hello World");
}
```

C

Calling a function with a string, in Java

```
public class Lecture2
{
   public static void printMessage( String message)
   {
      System.out.println(message);
   }

   public static void main(String[] args)
   {
      printMessage( "Hello World" );
   }
}
```

Java

Note: println() adds the \n, print() doesn't

- Everything inside a class
- All methods 'static'
- char* -> String
- printf -> System.out.println

Comparing the C and Java code…

```
public class Lecture2
{
    //              void printMessage( char* message )
    public static void printMessage( String message)
    {
        // printf("%s\n", message);
        System.out.println(message);
    }

    //              int  main(int argc, char* argv[])
    public static void main(String[] args)
    {
        printMessage( "Hello World" );
    }
}
```

- Everything inside a class

- All methods ‘static’
- char* -> String
- printf -> System.out.println

Aside: some Java conventions

- Use capital first letter for **class** names
  - E.g. String, Array, Book, Plane, Car, etc
- Use lower case first letter for **variable** names and **method / function** names
  - E.g. i, counter, computer, input, etc
- When names have multiple words, capitalise the first letter of subsequent words
  - E.g. myComputer, largeArray, etc
  - Or class names: BookArray, SetOfCars, …
- Braces:
  - Java style guide says to add open brace to end of line, and put closing brace on a new line
  - I usually (except for space reasons on slides) put the open brace on a new line, to aid in debugging (matching the braces)
  - Whichever you do, please be consistent (even if I use both)

Data types in Java

- Basic data types
  - int, float, boolean, double, long, char, etc
  - 'char' in Java is a 2 byte number to allow for extended character sets
  - 'byte' is a single-byte value
- Objects – more on these later – they are key to OO!
  - We use **object references** to refer to objects
  - **Object references** act like C/C++ **struct pointers**
  - Need to use new (or a function which does so) to create the object (like malloc() in C for pointers to structs)
  - Assigning one object reference to another means 'make it refer to (point at) the *same* object' (NOT copy the object!)
  - Passing an object reference into a function, or returning one, means referring to the same object, NOT creating a new one (only get a new object if you use new). **Think 'C pointer!'**

Strings and Numerical Classes

- String class (built into language)
  - `String` class exists for strings (a set of multi-byte characters, e.g. "Hello")
  - Strings are objects but they have extra some built-in functionality
  - **String literals** are `String` objects (no need to use new), e.g. "Hello"
  - + operator exists only for String objects, not other objects
    - Creates a new String by concatenating two existing strings (no 'new' is used)
  - Passed around like objects – by reference rather than making copies
  - Note: Strings are **immutable!**
    Use a `StringBuffer` object (or StringBuilder object) if you need to *change* a string's contents
- Basic data types (boolean, int, float, char, double, etc) are **not** objects, but can often be treated as if they were
  - Will explain in a later lecture why this is needed and how it is done
  - It wraps up the basic data type value in an object, silently converting to and object or back again
  - So ***sometimes*** you can treat basic types as if they are objects

Java: string concatenation

- **You can use + to concatenate strings in Java**
- Note: `println` already adds the final \n
  - Use `print` if you don't want it

C

```c
void printMessage2(char* message)
{
   char* mymessage = "Goodbye";
   printf("%s\n%s\n", message, mymessage );
}
```

Java

```java
public static void printMessage2( String message)
{
   String mymessage = "Goodbye";
   System.out.println( message + "\n" + mymessage );
}
```

A C program : concatenating strings

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
void printMessage(char* message)
{
   char* mymessage = "Goodbye";
   char* combined = malloc(strlen(message) +
                  strlen(mymessage) + 1);
   strcpy(combined, message);
   strcpy(combined + strlen(message), mymessage);
   printf("%s\n\n", combined);
   free(combined);
}
```

Java can avoid all of this work…

Java: concatenated strings

```
public static void printMessage2(
      String message)
{
  String mymessage = "Goodbye";
  String combined = message + mymessage;
  System.out.println( combined );
}
```

- String literals (e.g. “Goodbye”) are *String objects*
- Can use + to concatenate strings to get a **new** String
  - no need to allocate memory as in C, no need for ‘new’
- Note: Strings are ***immutable*** objects (unchangeable)

A C program : variables and conditions

C

```
void printMessage3()
{
  int i = 5; // Variable declaration
  int j = 6;
  if ( i != j ) // Condition
    printf("%d != %d\n", i, j );
}
```

- Concatenating strings is a pain in C – we rely on printf to do it here, formatting the output

Java: variables and conditions

```
public static void printMessage3()
{
   int i = 5; // Variable declaration
   int j = 6;
   if ( i != j ) // Condition
     System.out.println("" + i + " != " + j );
}
```

Java

- + can concatenate strings
- + can also append basic types (int, float, double) to strings
  - Can also add objects – calls the toString() method on the object - later
- Hint: start with a String to be sure it will work…
  - E.g. the empty string I used here : ""

Adding Strings and basic data types

- String addition will concatenate strings
- Numeric addition creates another number
- Numbers can be converted to strings if necessary

- What do you think is the output of the following?

```
public static void main(String[] args)
{
        System.out.println( "" + 4 + "A" );
        System.out.println( 4 + "A" );
        System.out.println( "" + 4 + 5 + "A" );
        System.out.println( 4 + 5 + "A" );
}
```

Simple rule : + works left to right

```
public static void main(String[] args)
{
        System.out.println( "" + 4 + "A" );
        System.out.println( 4 + "A" );
        System.out.println( "" + 4 + 5 + "A" );
        System.out.println( 4 + 5 + "A" );
}
```

Output:

4A	Starts with a String so it's fine
4A	int + String must be a String, can't be an int
45A	Starts with a String so it's fine
9A	int+int+String is (int+int)+string: (4+5)+"A"

Reason: It goes left to right in evaluating the expression

A C program : command line arguments

```
#include <cstdio>

int main(int argc, char* argv[])
{
  for (int i = 0; i < argc; i++)
    printf("Param %d is %s\n", i, argv[i]);
}
```

- argc is the count
- argv is an array of char* (pointers)
- argv[0] is the program name/path
- Note the [] comes after the variable name in C

Java: command line arguments

```
public static void main(String[] args)
{
  for (int i = 0; i < args.length; i++)
    System.out.println("Param "
          + i
          + " is "
          + args[i] );
}
```

- Note the [] in the type comes before the variable name
- **Arrays have a length**: use .length, as if it was a 'struct' member
- **There is no 'filename' as the first argument**
  - In C index 0 is the executable name that you executed

Putting it all together…

A C program to modify a string

- Assume that we have a C program which will go through a string and increase the value of each character by 1 (e.g. ‘a’ becomes ‘b’, ‘b’ becomes ‘c’ etc)
- We either use a command line argument for the string, or we have a default (if no command line argument is provided)

1. Get the string to use, in a format we can modify
2. Loop through the characters, changing them
3. Print the modified string

A C program : string modification

```
int main(int argc, char* argv[])
{
   const char* word = "Hello World"; // String literal
   if (argc >= 2)
      word = argv[1];

   // We are not allowed to alter the word ourselves!
   char* copy = (char*)malloc(strlen(word) + 1);
   strcpy(copy, word);

   int i = 0;
   while (copy[i]) // While character is not 0
   {
      copy[i]++;
      i++;
   }

   printf("Final: %s\n", copy);
   free(copy);
}
```

Important:
You don't need to know C for this module, but I hope you can understand this so that we can make the comparison to Java

C command line arguments

```
int main(int argc, char* argv[])
{
   const char* word = "Hello World";
   if (argc >= 2)     // Need 2 : the filename and an extra one
      word = argv[1]; // the first arg is argv[1] not argv[0]

   // We are not allowed to alter the word ourselves!
   char* copy = (char*)malloc(strlen(word) + 1);
   strcpy(copy, word);

   int i = 0;
   while (copy[i]) // While character is not 0
   {
      copy[i]++;
      i++;
   }

   printf("Final: %s\n", copy);
   free(copy);
}
```

The first part shouldn't be too hard:

- command line arguments change
- put it in a class
- use a String

Java-ise it… command line args

```
public static void main(String[] args)
{
   String word = "Hello World";      // Was const char*
   if (args.length >= 1)      // Now it is 1 we need, not 2
      word = args[0];         // Element 0 is the first one

   // We are not allowed to alter the word ourselves!
   char* copy = (char*)malloc(strlen(word) + 1);
   strcpy(copy, word);

   int i = 0;
   while (copy[i]) // While character is not 0
   {
      copy[i]++;
      i++;
   }

   printf("Final: %s\n", copy);
   free(copy);
}
```

Changed to use the Java command line arguments

C program: copy the word

```
public static void main(String[] args)
{
   String word = "Hello World";
   if (args.length >= 1)
      word = args[0]; // Element 0 is the first one

   // We are not allowed to alter the word ourselves!
   char* copy = (char*)malloc(strlen(word) + 1);
   strcpy(copy, word);

   int i = 0;
   while (copy[i]) // While character is not 0
   {
      copy[i]++;
      i++;
   }

   printf("Final: %s\n", copy);
   free(copy);
}
```

Now we need space for a modifiable copy of the string

String is immutable

A Java version: mutable copy

```
public static void main(String[] args)
{
   String word = "Hello World";
   if (args.length >= 1)
      word = args[0]; // Element 0 is the first one

   // We are not allowed to alter the word ourselves!
   StringBuffer copy = new StringBuffer(word);

   int i = 0;
   while (copy[i]) // While character is not 0
   {
      copy[i]++;
      i++;
   }

   printf("Final: %s\n", co
   free(copy);
}
```

We can use a StringBuffer for the modification

A StringBuffer is a ‘proper’ class

Create an object using ‘new’

Can provide parameters to it too

Applying functions/methods to objects

- Functions are applied to objects
  - Expressed as **`object.function()`**

- **`ob.f(x)`** means apply the function **`f()`** on the object referenced by **`ob`** and pass it the parameter x
  - E.g. jason.kick(football), door.open() , jason.sitOn(chair)

- A class defines what we can do with objects of that class
  - i.e. what functions (methods) exist
- **`StringBuffer`** lets us:
  - ask for string length : **`int length()`**
  - get a character from a position : **`char charAt(int pos)`**
  - change a character: **`void setCharAt(int pos, char c)`**
  - and many other things

C code: What does this bit do?

```
public static void main(String[] args)
{
   String word = "Hello World";
   if (args.length >= 1)
      word = args[0]; // Element 0 is the first one

   // We are not allowed to alter the word ourselves!
   StringBuffer copy = new StringBuffer(word);

   int i = 0;
   while (copy[i]) // While character is not 0
   {
      copy[i]++;

      i++;
   }

   printf("Final: %s\n", copy);
   free(copy);
}
```

Same code as before but removed blank line

Ready for the next bit…

What does this do?

Java code: charAt() and setCharAt()

```
public static void main(String[] args)
{
   String word = "Hello World";
   if (args.length >= 1)
      word = args[0]; // Element 0 is the first one

   // We are not allowed to alter the word ourselves!
   StringBuffer copy = new StringBuffer(word);

   int i = 0;
   while ( i < copy.length() ) // While not at end of string
   {
      char c = copy.charAt(i);
      copy.setCharAt( i, (char)(c + 1) );
      i++;
   }

   printf("Final: %s\n", copy);
   free(copy);
}
```

We use functions on the StringBuffer called *copy*

Java code: Casting an int to a char…

```
public static void main(String[] args)
{
   String word = "Hello World";
   if (args.length >= 1)
      word = args[0]; // Element 0 is the first one

   // We are not allowed to alter the word ourselves!
   StringBuffer copy = new StringBuffer(word);

   int i = 0;
   while ( i < copy.length() ) // While not at end of string
   {
      char c = copy.charAt(i);
      copy.setCharAt( i, (char)(c + 1) );
      i++;
   }

   printf("Final: %s\n", copy);
   free(copy);
}
```

Cast: (char) means convert to a char

A Java version

```
public static void main(String[] args)
{
   String word = "Hello World";
   if (args.length >= 1)
      word = args[0]; // Element 0 is the first one

   // We are not allowed to alter the word ourselves!
   StringBuffer copy = new StringBuffer(word);

   int i = 0;
   while ( i < copy.length() ) // While not at end of string
   {
      copy.setCharAt( i, (char)( copy.charAt(i) + 1 ) );

      i++;
   }

   printf("Final: %s\n", copy);
   free(copy);
}
```

We can do both in one line

Only one little bit left to change

```
public static void main(String[] args)
{
   String word = "Hello World";
   if (args.length >= 1)
      word = args[0]; // Element 0 is the first one

   // We are not allowed to alter the word ourselves!
   StringBuffer copy = new StringBuffer(word);

   int i = 0;
   while ( i < copy.length() ) // While not at end of string
   {
      copy.setCharAt( i, (char)( copy.charAt(i) + 1 ) );

      i++;
   }

   printf("Final: %s\n", copy);
   free(copy);
}
```

Finally the output…

```
public static void main(String[] args)
{
   String word = "Hello World";
   if (args.length >= 1)
      word = args[0]; // Element 0 is the first one

   // We are not allowed to alter the word ourselves!
   StringBuffer copy = new StringBuffer(word);

   int i = 0;
   while ( i < copy.length() ) // While not at end of string
   {
      copy.setCharAt( i, (char)( copy.charAt(i) + 1 ) );

      i++;
   }

   System.out.println( "Final: " + copy );
   // free() not needed!
}
```

Changing of output as before – we can add the object

No need to free memory in Java – jvm does it

Important note

- char in Java is a two byte number for an extended character
  - Based on original Unicode definition
  - https://en.wikipedia.org/wiki/List_of_Unicode_characters
  - I treated it the same way as a C char
- Both Java String and Java char types support Unicode characters
  - Don't assume that they are ASCII or only one byte
  - Many functions and classes convert the type internally though
    - So there are facilities to read and write ASCII text files, for example
- I actually cheated here to allow a simple porting of code
  - I happen to know that the first 127 printable ASCII characters match the Unicode character codes so I basically treated the character as it if was ASCII (as in C)
    - It worked **only** because the codes are the same in this range
  - This code may break with characters outside of the range of the printable ASCII characters
    - Probably not a good thing to do in general anyway
    - But it allowed me to convert C to Java code for Strings without a lot of work – to demonstrate the similarities and differences

Key messages

- You can fairly easily convert your C programs to Java
  - You need to modify anything which uses pointers – to avoid needing them
- We have used some objects of simple classes
  - E.g. String (sort-of object), StringBuffer (a class of real objects)
- Classes/objects are a grouping of data and some functions/methods to act upon it
  - Classes are the plan for the objects
  - Objects are the sets of data at runtime, that functions can act upon

What next?

- Try lab 1 to get started. Watch the videos to get started
  - Lab exercises 1 are basically just a getting started guide and help you to install or run Eclipse, but encourage you to try the code from Lecture 2.
- Coursework 1 is available – do lab 1 first, then it's trivial to do
- Try to do lab 2, and ask questions in the lab time
  - **Labs are entirely optional, but a good time to ask questions**
  - **No attendance monitoring, as they are optional**
  - Lab exercises 2 encourages you to go through a different exercise yourself, based upon what you saw in this lecture
- Next lecture we will look at some more Java code, what we are really doing, then at how to identify *appropriate* objects in our own programs, and the basics of object oriented design